

Test Report : GoQuant – GoTrade Website

Application Under Test: GoTrade Website

System Test Period: October 06-29, 2025

Tester: Simon Leonard D

Testing Framework: Playwright with TypeScript

Browsers Tested: Chromium 118.0, Firefox 119.0, WebKit 17.0

Executive Summary

This report presents the findings from manual and automated testing of GoQuant's GoTrade Website conducted over a period of 24 days of intensive testing. During this period, I executed 78 carefully designed test cases spanning around functional validation, User Interface testing, security validation and edge case scenarios. The automation and manual testing process revealed concerns about the application's readiness on production deployment with distinct 13 issues identified across all severity levels.

Performance testing revealed concerning scalability limitations, with the application showing exponential performance degradation when performing continuously automated trade for 50 times. In styling, the classes are directly mentioned in the `<div>` in the instead of assigning it to a unique variable which helps in reducing the time taken to find the unique element for a `<div>` and it helps in avoiding multiple selectors. The website hangs when performing operations and becomes unresponsive.

The testing identified 4 critical issues requiring immediate attention, 4 high-priority concerns that should be addressed before launch, 3 medium-priority improvements for enhanced user experience, and 2 low priority cosmetic issues. My primary recommendation is to postpone production deployment until the critical and high-priority issues are resolved, particularly focusing on authentication security and data integrity mechanisms.

Moving forward, I recommend implementing performance testing with multiple runs on the same major actions, load testing with multiple users actions and UI testing to avoid overlapping. Continuous integration testing can be performed to avoid these limitation in the earlier stage of deployment.

Testing Methodology

Testing Approach

A comprehensive manual testing has been carried out with priority of user journeys and failure points within GoTrade website. This approach was designed to find the major parts of the application where the failures cause dissatisfaction to the user. Following

manual testing, automated testing was performed in order to automate the test cases which must be executed every time in order to maintain smooth functioning of basic but essential flow. The methodology was structured around four phases and each phase depends on the previous phase results.

PHASE 1:

Smoke testing was performed to ensure that the application basic functionality was operational across different browsers and devices. This phase includes basic operations such as application loading, login & logout operations, basic navigation, basic account creation in a straight-forward manner. The testing is to observe and verify the basic functionalities of the application is working fine and move forward with the extensive and automate testing phase.

PHASE 2:

Extensive and Complex operational testing was performed in the application to ensure that application is bug free and no major set backs arise after production deployment. Automated testing was designed around basic and complex workflows and business logics. Automated testing includes complete automation of user journey from login to account manipulations (creation, deletion, modification) to placing trade to check related graphs and charts. A careful observation on the data persistence and application behaviour is noted.

PHASE 3:

Performance and load testing was performed following automated testing. These tests are performed to ensure that the application can withstand large number of users and large number of operations simultaneously among multiple users.

PHASE 4:

API Testing was performed to ensure the application can be accessible via rest calls and works as expected.

Test Case Selection Rationale and In-Depth Analysis

The selection of test cases was driven by a thorough risk assessment of the GoTrade application, considering both business impact and technical complexity. I developed 78 distinct test cases organized into six primary categories, each addressing specific aspects of application quality and user experience.

Authentication and Security Test Cases (7 tests): These tests were prioritized as the highest risk category due to the sensitive nature of user data and the potential for security breaches. I designed test cases to validate not only successful authentication flows but also to probe for common security vulnerabilities such as unauthorized access attempts, direct link access, broken link. The test cases included scenarios for invalid user-email, invalid password, direct access of home page without logging in.

Core CRUD Operations Test Cases (60 tests): As the primary functionality of the application, CRUD operations received the most comprehensive test coverage. These test cases includes account creation, updation and deletion under various conditions and user contexts in admin screen. The test cases includes Valid trade creation, trade with

incomplete values and trade with '-' values, performing cancel working orders, edge kill and cancel open positions. Special attention was paid to edge cases such as creating tasks with symbols, performing trade with empty values.

User Interface and Experience Test Cases (6 tests): These test cases focused on ensuring the application provides a consistent and accessible user experience across different devices, browsers, and user capabilities. Tests are developed to validate responsive design behaviour at various viewport sizes, cross browser rendering consistency. The visual regression testing component used automated screenshot comparison to detect unintended UI changes such as chart, dropdown, alert messages.

Performance and Scalability Test Cases (2 tests): Understanding that trading applications must handle varying data loads efficiently, Created test cases specifically designed to evaluate application performance under different stress conditions such as multiple user trading, multiple continuous order placements.

API Test Cases (3 tests): To ensure rest calls, API logins are performed, But due to limitations on the application unable to proceed.

Tools and Techniques Employed

The testing framework was built around Playwright as the primary automation tool, chosen for its robust cross-browser support, excellent handling of modern web applications, and comprehensive API for functional testing. Implemented the Page Object Model design pattern to ensure test maintainability and reusability, creating separate page objects for each major application screen and component.

Test data management was stored in a typescript file and imported into the tests where they are necessary. This approach is to make the data consistent across different operations and not dependent on results of previous test cases.

Selector management for every major application screen was implemented and the locators are stored in a separate file with the screen name and can be reused across multiple screen where ever they are necessary.

Constant words on every major application screen was stored in a unique file name representing the screen in a separate folder and can be reused across multiple screen where they are necessary.

Visual representation testing was implemented using playwright's in-built screenshot capabilities, with screenshot algorithms to detect pixel difference between expected and the actual UI. This can be used to find the difference in the pixel across each browsers. To overcome the UI pixel difference across each browsers the images are converted into a common pixel size. The screenshot comparison code has been stored in a common folder in order to access across multiple tests across different pages.

API testing was conducted in order to examine whether the application can be accessible via rest calls. The results are cross verified using 'POSTMAN'.

Challenges Encountered and Solutions Implemented

Throughout the testing process, I encountered various technical challenges during automation of test cases and resolved the challenges in the most effective & efficient way possible.

The most significant challenge involves handling flaky network requests which causes the test cases to fail due to timeout. This issue was addressed by adding necessary waits between navigation and actions. These custom or explicit wait step allows the asynchronous steps to complete after a defined time so the execution proceeds without any interruptions.

In API testing, For login, the API request does return true for the successful case but in case of unsuccessful cases the 'POST' request returns 405 even though in the network tab it returns the response 404 but due to API endpoint doesn't accepting post requests, even though the credentials are accepted but the request is rejected.

The screenshot comparison among different browsers showed inconsistencies across the pixel of the image and failed in automation testing. It was addressed by converting the pixel into equal size by sharp library.

Detailed Findings

Critical Issues

CRIT-001 : The GoMarket screen is unresponsive

Severity: Critical

Browser: All

Description: The GoMarket screen hangs up frequently

Steps to Reproduce:

1. Login into GoTrade
2. Navigate to GoMarket screen
3. Wait for sometime

Expected behaviour : The GoMarket screen should not hang and work properly

Original behaviour : The GoMarket screen hangs and not responding

Evidence : Screenshot saved as evidence/CRIT_001.png

CRIT-002 : Irrelevant trade details are visible in trade table

Severity : Critical

Browser : All

Description : In Order table, The account or exchange details which are not related to the user are displaying.

Steps to Reproduce :

1. Login to GoTrade
2. Navigate to GoTrade screen
3. Scroll down to trade table

Expected behaviour : The trade table should display the details regarding the orders which happens only in the account.

Original behaviour : The trade table is displaying the details of trade which are not placed by the account holder.

Evidence : Screenshot saved as evidence/CRIT_002.png

CRIT-003 : Modifying an account is not working properly

Severity : Critical

Browser : All

Description : In admin page, modifying an account details throws credentials() error

Steps to reproduce:

1. Login to GoTrade
2. Navigate to admin screen
3. In account trade, Click modify button of an account
4. Change the account name, and enter api and secret keys
5. Click modify button

Expected behaviour : The details must be updated

Original behaviour : Throws delete_credentials() error

Evidence : Screenshot saved as evidence/CRIT_003.png

CRIT-004 : The decay factor allows '-'

Severity Critical

Browsers : All

Description : The platform allows trade even when the decay factor is mentioned as '-'

Steps to Reproduce:

1. Login into GoTrade
2. In GoTrade Page, Enter Quantity as 1.
3. Enter duration as 10.
4. Enter decay factor as '-'.
5. Click trade button

Expected behaviour : Trade should be rejected

Original behaviour : Trade is accepted

Evidence : Screenshot saved as evidence/CRIT_004.png

High Priority Issues

High-001 : The spread graph is not displaying properly

Severity : High

Browsers : All

Description : The spread graph is not displaying

Steps to Reproduce:

1. Click on Trading dropdown
2. Click on GoTrade option from dropdown

Expected behaviour : Graph should be displayed

Original behaviour : Graph is not displayed

Evidence : Screenshot saved as evidence/HIGH_001.png

High-002 : Consolidated graph is not displaying

Severity : High

Browsers : All

Description : After turning on consolidated toggle, the order book, spread and latest trade cards are not loading properly

Steps to Reproduce :

1. Click on Trading dropdown
2. Click on GoTrade option from dropdown
3. Click on the consolidated toggle in the order book card.

Expected behaviour : The consolidated graph should be displayed

Original behaviour : The consolidated graph is not displayed

Evidence : Screenshot saved as evidence/HIGH_002.png

High-003 : Last trade graph is displaying incorrectly

Severity : High

Browsers : All

Description : In the last trade graph, the execution time is same for all the orders

Steps to Reproduce :

1. Click on Trading dropdown
2. Click on GoTrade option from dropdown
3. Check the Last trade graph

Expected behaviour : The Last trade graph should have execution time differently based on execution

Original behaviour : The last trade graph, the execution time is same.

Evidence : Screenshot saved as evidence/HIGH_003.png

Medium Priority Issues

MED-001 : Validation message is displaying incorrectly in TakeProfit card

Severity : Medium

Browsers : All

Description : The validation message of duration is displaying incorrectly below decay factor textbox

Steps to Reproduce :

1. Click on Trading dropdown
2. Click on GoTrade option from dropdown
3. Click on quantity and enter 1
1. 4. Click on the duration textbox and enter 1
4. Click on decay factor and enter 1
5. Click on the TP checkBox
6. Click on trade

Expected behaviour : The validation message of duration should be displayed below duration textbox.

Original behaviour : The validation message of duration is displaying below decay factor textbox

Evidence : Screenshots saved as evidence/MED_001.png

MED-002 : Algorithm type comparison is not showing values

Severity : Medium

Browsers : All

Description : The Algorithm type comparison chart is not showing any values

Steps to Reproduce :

1. Click on Trading dropdown
2. Click on GoTrade Option from the drop down
3. Click on discovery mode in trade card
4. In Instrument type, select SPOT
5. In Base Asset, select BTC
6. In Quote Asset, select USD

Expected behaviour : The pre-trades details should be visible

Original behaviour : The pre-tades details are not visible

Evidence : Screenshot saved as evidence/MED_002.png

MED-003 : Wrong account name message is displayed

Severity : Medium

Browsers : All

Description : After typing a name "ww" in account name text box, it displays please provide the account name. The reason to it is the length of the name, of minimum 3 .

But instead of displaying minimum length is 3 it is displaying please provide account name

Steps to Reproduce :

1. Go to Accounts
2. Click on admin option
3. click on add account
4. Enter "ww" in Account name box

Expected behaviour : The display should be the minimum length for account name is 3.

Original behaviour : It displays "Please provide the account name"

Evidence : Screenshot saved as evidence/MED_003.png

Low Priority Issues

LOW-001 : Incorrect page number

Severity : Low

Browsers : All

Description : The page should show total no.of.pages available instead of 0

Steps to Reproduce :

1. Click on Trading dropdown
2. Click on GoTrade Option from the drop down
3. Scroll down to get "working orders" trade table

Expected behaviour : In the working orders table, The page number should be 1 of 1.

Original behaviour : In the working orders table, the page number is 1 of 0.

Evidence : Screenshot saved as evidence/LOW_001.png

LOW-002 : Discovery toggle overlaps on symbol

Severity : Low

Browsers : All

Description : The discovery mode toggle icon is overlapping on the symbol after perform some operations

Steps to Reproduce :

1. Click on the Market dropdown
2. Click on GoMarket
3. Enable discovery mode
4. In Instrument type, select spot
5. Select symbol, BTC

Expected behaviour : The toggle should not be rearranged or overlap

Original behaviour : The toggle is rearranged and overlapped

Evidence : Screenshot saved as evidence/LOW_002.png

Technical Analysis

Performance Observations

The performance analysis reveals the application might slow or crash during real-world usage. The home page takes about 10 seconds on several occasions to load. The graphs and chart are not loaded even after home page loaded. The market page hangs up frequently which results in unresponsiveness.

Browser Compatibility Issues

By testing across different browsers, the main issue is the pixel difference of images and content loading of the pages. Especially in firefox, the page takes some extra time to load.

Code Quality Observations

The codebase analysis states the error message or popup validations can be more fine tuned. Every css element should have an id and unique name and unique locator tag.

Test Execution Summary

The comprehensive test execution revealed significant quality concerns across all testing categories, with an overall pass rate of 84% indicating that the application requires improvements before production deployment. Of the 78 test cases executed, 72 passed successfully while 6 failed, with no tests skipped due to technical limitations. The entire test suite completed execution in 9 minutes and 17 seconds, demonstrating efficient test automation implementation.

The test results varied significantly across different functional areas.

1. Authentication testing revealed 3 of 3 tests passing successfully.
2. Link Verification (Security) testing revealed 4 of 4 tests passing successfully.
3. CRUD operation testing revealed 56 of 60 tests passing successfully.
4. User Interface and Experience testing revealed 6 of 6 tests passing successfully.
5. Performance and Scalability testing revealed 2 of 2 tests passing successfully.
6. API testing revealed 1 of 3 tests passing successfully.

Browser-specific analysis revealed varying levels of compatibility, with Chromium browsers showing the best performance at 92% pass rate, followed by Firefox at 74%, and WebKit at 83%. These differences suggest that the application was primarily developed and tested using Chromium-based browsers, resulting in suboptimal experiences for users of other browser platforms.

Recommendations for Improvement

Based on the comprehensive testing results and identified issues, I recommend a phased approach to addressing the application's quality concerns. The recommendations are prioritized by risk level and potential impact on user experience and network lag, network load.